

Cuando el malware sospecha que le estás observando: mi TFM sobre detección de entornos virtualizados

Si alguna vez has abierto un archivo sospechoso dentro de una máquina virtual "por si acaso", has hecho exactamente lo mismo que hacemos a diario en el análisis de malware: aislar el bicho en una jaula de cristal para verlo comportarse sin que pueda hacer daño de verdad. El problema es que ese malware, muchas veces, también sabe que está en una jaula de cristal. Y si lo sabe, se hace el tonto.

De eso trata mi Trabajo Fin de Máster: de cómo el malware descubre que está siendo analizado en una máquina virtual o en un sandbox, y de qué podemos hacer para que le resulte más difícil darse cuenta.

La idea de partida, explicada sin tecnicismos

Cuando analizamos software peligroso, casi nunca lo ejecutamos en nuestro propio ordenador. Usamos máquinas virtuales: ordenadores "de mentira" que corren dentro de uno real, y que podemos borrar y volver a crear en segundos si algo sale mal. Es la manera lógica y segura de estudiar malware sin arriesgar nada.

El truco es que crear ese ordenador de mentira deja huellas. VirtualBox, VMware o Hyper-V instalan sus propios controladores, herramientas de "invitado" (Guest Additions), identificadores de hardware particulares y hasta direcciones de red con un sello de fábrica reconocible. Son detalles que un usuario normal nunca ve, pero que un programa puede consultar en un instante.

Un malware bien hecho hace justo eso antes de actuar: mira a su alrededor, comprueba una docena de pistas y, si algo le chirría, decide que probablemente está siendo vigilado. Entonces puede quedarse quieto, fingir ser inofensivo, o simplemente no ejecutar su carga dañina. Resultado: el analista cree que ha estudiado el malware, pero en realidad solo ha visto la versión "educada" que el malware reserva para cuando sospecha que le observan.

Por qué merece la pena estudiar esto

Este comportamiento no es una curiosidad técnica menor: es un problema real para cualquiera que trabaje en análisis de malware, sea en una empresa de ciberseguridad, en un laboratorio o simplemente aprendiendo. Si no entendemos qué pistas delatan a nuestro propio laboratorio, corremos el riesgo de sacar conclusiones erróneas sobre una muestra que en realidad nunca llegó a mostrar su comportamiento real.

Así que mi trabajo parte de una pregunta muy práctica: ¿qué señales concretas usa el malware para sospechar que está virtualizado, y cuáles de esas señales podemos reducir o camuflar?

Cómo lo he organizado: quince familias de pistas

En vez de mirar técnica por técnica sin ningún orden, agrupé todas las formas de detección en quince "familias", según el tipo de pista que explotan. Por ejemplo:

- **Sistema de archivos y Registro de Windows:** buscar archivos o claves que solo existen si hay software de virtualización instalado (el clásico `VBoxGuest.sys` de VirtualBox es un ejemplo de manual).
- **Procesos en ejecución:** comprobar si hay herramientas de análisis abiertas, como un depurador.
- **CPU y hardware:** preguntar directamente al procesador quién es el fabricante (una instrucción llamada `CPUID` puede llegar a responder literalmente "VBoxVBoxVBox", que es tan evidente como firmar con tu propio nombre).
- **Temporización:** medir si ciertas operaciones tardan un tiempo "raro", típico de entornos virtualizados o de análisis instrumentado.
- **Interacción humana:** comprobar si hay movimientos de ratón o pulsaciones de teclado recientes, algo que un sandbox automatizado rara vez simula bien.
- **Red, firmware, objetos internos del sistema, artefactos visuales de las herramientas de virtualización,** y varias más.

Cada familia se estudió con el mismo método: identificar qué comprueba exactamente el malware, reproducirlo con herramientas educativas y de pruebas de concepto —principalmente *Pafish* y *al-khaser*, dos proyectos pensados justamente para esto y que no son malware real—, y comprobar si esa comprobación decide realmente algo o es solo ruido.

El hallazgo más interesante

No todas las pistas valen lo mismo. Las señales muy explícitas y ligadas a un fabricante concreto —una cadena de texto del procesador, un identificador de dispositivo, una clase de ventana de una herramienta de virtualización— son pruebas casi irrefutables: si aparecen, es porque el entorno es virtual, sin lugar a duda razonable.

En cambio, las señales "indirectas" —poca memoria RAM, un único núcleo de CPU, ausencia de actividad de ratón, tiempos de respuesta algo lentos— son solo indicios. Pueden darse en una máquina virtual, pero también en un portátil modesto y apagado hace un rato. Son pistas heurísticas, con un margen real de error, y por eso el malware que las usa en solitario puede llegar a equivocarse (y de hecho lo hace).

¿Se puede hacer algo al respecto?

Sí, y esta es la parte que más me gusta del trabajo: también probé a "limpiar" una máquina virtual, quitando o disimulando muchas de esas huellas (desinstalando las Guest Additions, cambiando identificadores de hardware, borrando claves de Registro residuales, modificando la dirección MAC, etc.) y volví a pasar las mismas pruebas antes y después.

El resultado es realista y honesto: la sanitización reduce muchísimo el número de artefactos visibles, pero no los elimina todos. Algunas diferencias, especialmente las relacionadas con la

temporización, son muy difíciles de disimular por completo porque tienen que ver con cómo funciona la propia virtualización a bajo nivel. Es una victoria parcial, no una solución mágica, y creo que es importante decirlo así en vez de vender un resultado perfecto que no sería cierto.

Qué me llevo de todo esto

Más allá de los datos concretos, el TFM me ha dejado una idea central que quiero destacar: encontrar un artefacto sospechoso no es lo mismo que tener una conclusión. Un ordenador real puede tener poca RAM. Un servidor puede tener Hyper-V activo sin ser una máquina de análisis. La fuerza de una detección —tanto la del malware como la nuestra— está en cruzar varias señales coherentes entre sí, no en fiarlo todo a una sola comprobación.

Y desde el punto de vista educativo, todo el trabajo se apoya en pruebas de concepto controladas, aisladas y sin usar malware real salvo cuando fue estrictamente necesario. La idea es que estos mismos ejercicios puedan servir para que otra persona que empiece en este mundo entienda, paso a paso, cómo piensa un programa que intenta averiguar si le están vigilando... y cómo podemos ponérselo un poco más difícil.

Este artículo resume, en lenguaje llano, el Trabajo Fin de Máster "Detección de entornos virtualizados por malware", de María Sonia Salido Fernández (Máster en Análisis de Malware y Reversing, ENIIT, 7ª Edición, agosto de 2026).